

# Inhaltsverzeichnis

|                                      |   |
|--------------------------------------|---|
| 1. Assoziationen .....               | 1 |
| 1.1. Beziehungsarten .....           | 1 |
| 1.1.1. Kopplung .....                | 2 |
| 1.1.2. Kohäsion .....                | 3 |
| 1.2. Aggregation & Komposition ..... | 4 |
| 1.2.1. Aggregation .....             | 4 |
| 1.2.2. Komposition .....             | 4 |
| 1.3. Navigierbarkeit .....           | 5 |
| 1.4. One-to-One-Assoziation .....    | 6 |
| 1.5. One-to-Many-Assoziation .....   | 6 |
| 1.6. Many-to-Many-Assoziation .....  | 7 |

## 1. Assoziationen

Klassen existieren fast nie unabhängig, denn Software besteht immer auch vielen Klassen und weiteren Datenstrukturen, die nur **gemeinsam** funktionieren und somit die ganze Software bilden.

### 1.1. Beziehungsarten

Klassen gehen also mit anderen Klassen viele sogenannte "**Beziehungen**" ein. Dabei gibt es verschiedene Arten von Beziehungen. Die wichtigsten sind:

- **Aufruf** (*dynamische Beziehung zur Laufzeit*), kann innerhalb einer Software sein oder zwischen ganzen Systemen!

```
public class App {  
    public static void main(String[] args) {  
        Train train = new Train("ICE 234");  
        train.depart();  
        // ....  
    }  
}
```

Die Klasse **App** **benutzt** bzw. **ruft** die Klasse **Train** auf

- **Vererbung, Realisierung** (*statische Beziehung zur Compilezeit*)

```
public class Train extends Vehicle {  
    // ...  
}
```

- **Besitz** (*statische Beziehung zur Compilezeit*) → wird (neben weiteren Varianten) oft auch als **Assoziation** bezeichnet.

```
public class Train {
    private Driver driver; ①
    private List<Wagon> wagons; ①
}
```

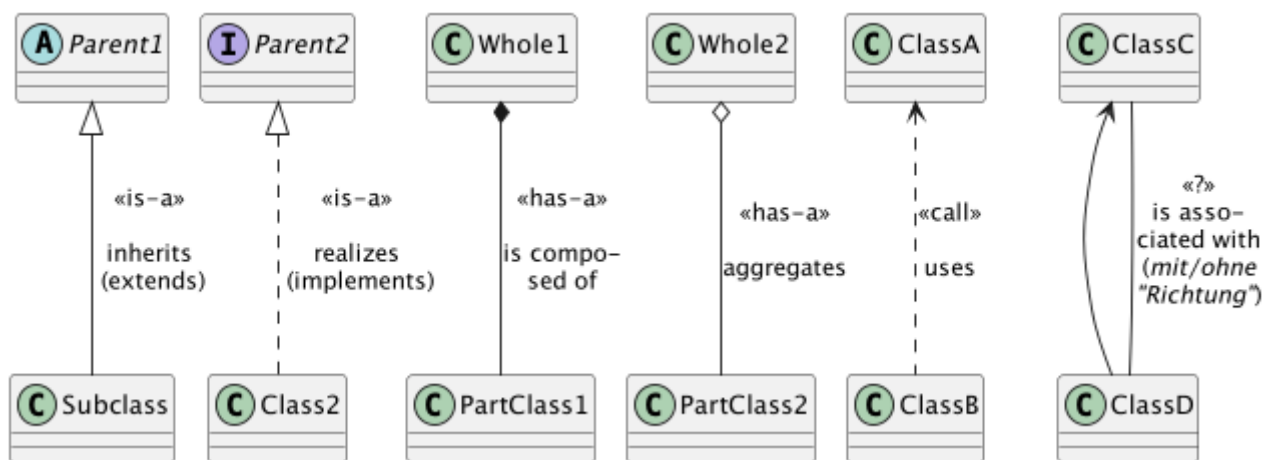
- ① Der Zug **hat** einen bzw. **besitzt** den Triebfahrzeugführer, außerdem **besteht** er aus Waggonen. Anders ausgedrückt, der Zug **setzt sich zusammen** aus *einem* Triebfahrzeugführer und *mehreren* Waggonen.

Vielfach wird bzgl. der Beziehungen zwischen Klassen zwischen folgenden zwei grundsätzlichen **Varianten** unterschieden:

1. **is-a** - Klasse A *ist eine* Klasse vom Typ B ⇒ "Vererbung"
2. **has-a** - Klasse A *hat eine* Klasse B ⇒ "Zusammensetzung"

Wie man im Beispiel erkennen kann, spielt es hierbei auch eine Rolle, in welcher "*Menge*" die anderen Klassen vorkommen. Das kann man unter dem Begriff "**Kardinalität**" zusammenfassen.

Zusammengefasst als Übersicht:



### 1.1.1. Kopplung

Art & Grad der Abhängigkeit **zwischen** (einzelnen) Softwareelementen, insb. Klassen.

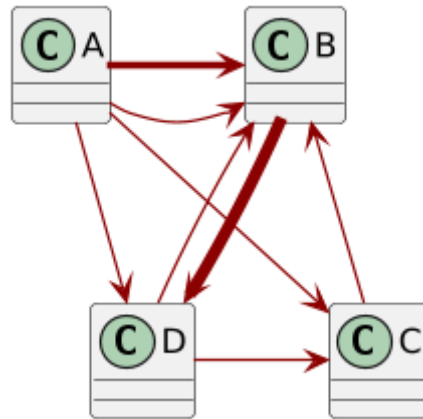


Figure 1. Kopplung

Die **Kopplung** im obigen Diagramm ist die (Art und) **Anzahl der Pfeile** zwischen den Klassen. Sie gibt an, wie stark die einzelnen Programmmodule (hier Klassen) von den anderen abhängig sind. Wechselwirkungen zwischen zwei Klassen/Objekten treten auf, weil es eine Kopplung gibt. Lose gekoppelte Programme sind flexibel und erweiterbar, je stärker die Kopplung, desto schwieriger wird die Änderbarkeit der Klassen.

❓ Wie könnte man starke oder schwache Kopplung im Quellcode, z.B. in einer Klasse erkennen?

### 1.1.2. Kohäsion

Grad des Zusammenhangs (z.B.) von Klassen **innerhalb** eines Moduls/Pakets.

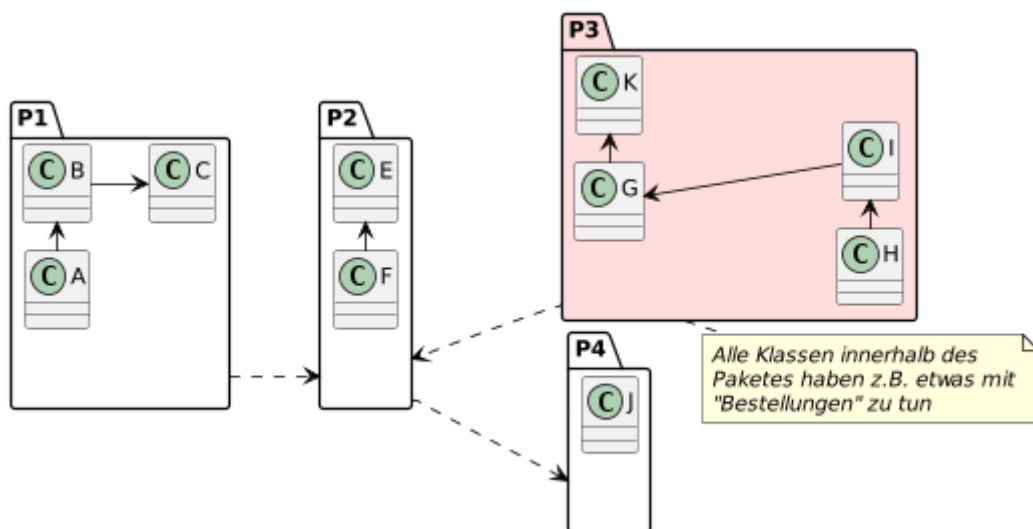


Figure 2. Kohäsion (hier z.B. im rot hinterlegten Paket)

**Kohäsion** misst, wie stark die einzelnen Klassen innerhalb eines Programmmoduls - z.B. eines Paketes - zusammenhängen, vor allem wie stark sie zu *einem/wenigen* Aspekt oder auch Zusammenhang gehören. Gut **strukturierte** Software führen oft zu stark zusammenhängenden Anwendungen.



Der Grad - also die Anzahl und Qualität - der Summe aller dieser Beziehungen zwischen den Klassen einer Software wird unter dem Begriffspaar **Kopplung &**

**Kohäsion** zusammengefasst ("**loosely coupled along with high cohesion**"). Ein allgemeines Qualitätsziel von Software ist bspw., einen niedrigen Grad von Kopplung der Komponenten zu erreichen. Je geringer die Kopplung ist, desto besser ist die Software änderbar. Gleichzeitig sollen die Klassen innerhalb eines Paketes/Moduls zur gleichen Fachlichkeit gehören!

### Beispiel:

Die **Organisation dieser LV** ist so aufgebaut. Es gibt viele einzelne Module, in denen Klassen enthalten sind, die eben zu diesem Modul bzw. zu dem jeweiligen Thema gehören, sie sind entsprechend "**hoch kohäsiv**".

Gleichzeitig haben die Module untereinander keine bzw. sehr wenige Abhängigkeiten. sie sind zu anderen Modulen nur "**lose gekoppelt**"!

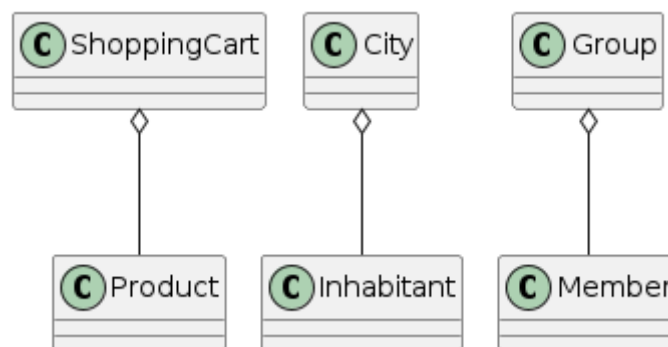
## 1.2. Aggregation & Komposition

Assoziationen können auch **qualitativ und semantisch** recht unterschiedlich sein. In diesem Zusammenhang sind **Aggregation** und **Komposition** von Bedeutung.

### 1.2.1. Aggregation

Die Aggregation, sowie auch die Komposition, ist eine Assoziation von **Teilen zu einem Ganzen**. Jedes Teil ist zu dem Ganzen mit einer Assoziation "**ist-Teil-von**" (part-of) verbunden.

Ein paar **Beispiele** dazu:



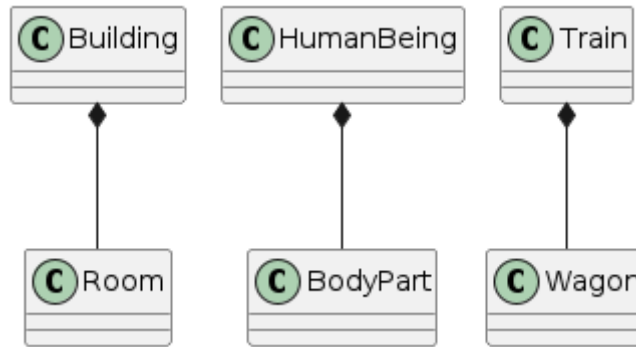
### Demo:

```
src/test/java/de/dhbw/demo/AssociationsDemoTest.demo1()
```

### 1.2.2. Komposition

Der Unterschied einer Komposition zur Aggregation ist, dass die Teile, die ein Objekt enthält, von dem Ganzen **existentiell abhängig** sind.

Auch dazu ein paar **Beispiele**:



Die Unterschiede sind also fein, aber wichtig.

❓ Es stellt sich die Frage, wie man so eine **"existenzielle Abhängigkeit"** im Code ausdrücken kann?

Eine Möglichkeit ist:

```

public class Document {
    private TableOfContent toc;
    public Document(TableOfContent inhaltsverzeichnis) { ①
        this.toc = inhaltsverzeichnis
    }
}
  
```

① Starkes **Indiz** für existenzielle Abhängigkeit: Das direkte Setzen der abhängigen Klasse und (oft auch) deren Initialisierung im oder über den *Konstruktor*!

Eine andere Idee ist in folgender **Demo**-Methode umgesetzt:

**Demo:**

```
src/test/java/de/dhbw/demo/AssociationsDemoTest.demo2()
```

## 1.3. Navigierbarkeit

Sobald Klassen durch Assoziationen verbunden werden, kommt der Aspekt der **Navigierbarkeit** (von Instanz zu Instanz) hinzu, d.h. man überlegt, welche Klasse welche als Attribut enthält und ob auch die assoziierte Klasse etwas über diese Beziehung weiß.

Letztlich drückt sich die Navigierbarkeit darin aus, ob assoziierte Klassen

- als **komplexes Attribut** → (2) realisiert sind oder
- nur als **primitiver "Referenz"-Wert**, z.B. die **id** der anderen Klasse → (3)

vorhanden sind (auch die "besitzende" Klasse hat in aller Regel selbst eine ID → \*(1)\*).

Am **Beispiel**:

```
public class Train {
```

```

public long id; ①
public Locomotive locomotive; ②
public long locomotiveId; ③
}
// eine Referenz als "virtuelle Beziehung"
public class Locomotive {
    public long id; <-----+
    public Train train;
}

```

❓ Überlegung: Ist es sinnvoll, die Klasse **Train** als Feld/Attribut der Klasse **Locomotive** zu haben?

## 1.4. One-to-One-Assoziation

Siehe Unit-Test:

```
src/test/java/de/dhbw/demo/AssociationsDemoTest.demo3()
```

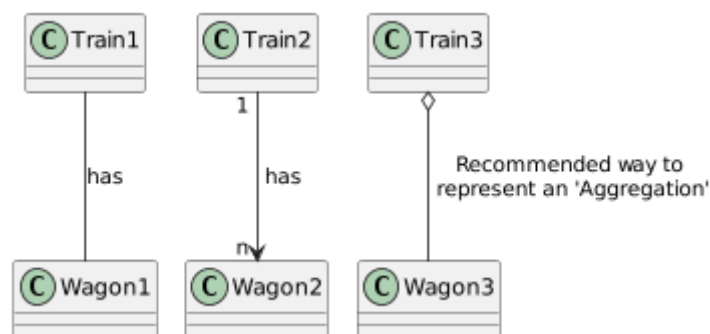
### Ein Beispiel

Jede natürliche Person besitzt **genau** einen Personalausweis, der Ausweis kann verloren gehen, die Person kann sterben, dann muss aber auch der Personalausweis vernichtet werden, das wurde oben bereits demonstriert.

Es geht hier also oft um die Überlegung zur **Semantik bzw. Qualität** einer Beziehung zwischen zwei abhängigen Objekten!

## 1.5. One-to-Many-Assoziation

Ein Beispiel für eine Eins-zu-Viele Beziehung:



Man beachte immer auch die Richtung der Pfeile, die in aller Regel etwas über die **Lesart** und **Navigierbarkeit** aussagt!

Zugehörige Unit-Tests zur **Demonstration**:

```
src/test/java/de/dhbw/demo/AssociationsDemoTest.demo4a()
```

```
src/test/java/de/dhbw/demo/AssociationsDemoTest.demo4b()
```

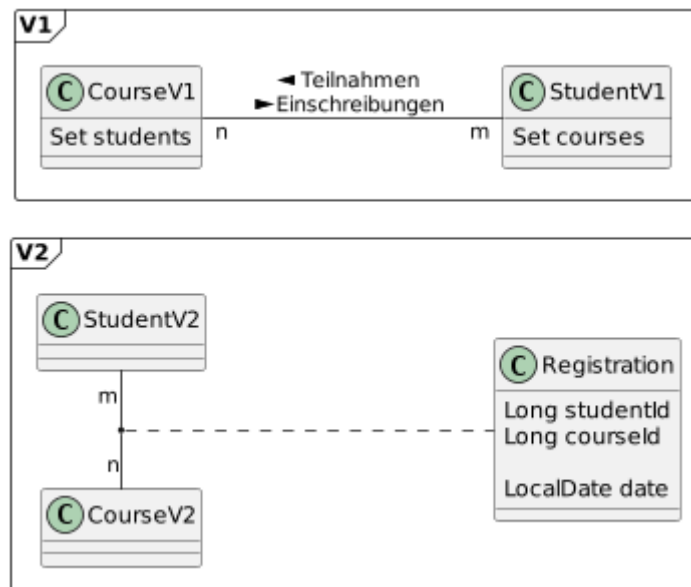
sowie Implementierungen für V1 und V2:

```
src/main/java/de/dhbw/demo/associations.n.*.java
```

→ Übung 1

## 1.6. Many-to-Many-Assoziation

Die folgenden zwei Varianten kommen am häufigsten vor:



Assoziationen zwischen Java Klassen werden manchmal auch als **Relationen** bezeichnet. Dieser Begriff stammt aus dem Umfeld der **Datenbanken**, meint aber das Gleiche. Dies ist ein optionales Thema und kann bei Gelegenheit eingeschoben werden.

### Übungen:

Die **Übungen** werden in Form von **Unit-Tests** in folgendem *Package* implementiert:

```
src/test/java/de/dhbw/exercises/AssociationsExerciseTest.java
```

### Übung 1

Erstelle zwei Klassen

- **Course** und
- **Student**

Der Kurs kann von **mehreren** Student:innen besucht werden. Setze diese Beziehung zwischen den beiden Klassen in der Klasse **Course** um.

## Übung 2 (optional)

### Umsetzung des Prinzips **Information Hiding**

In dem folgenden Paket finden sich die Klassen aus der Demonstration:

```
src/test/java/de/dhbw/demo/associations.nm.v2
```

Diese Klassen enthalten aber noch keine Methoden!

Füge die zu den Attributen gehörenden **getter** und **setter** Methoden hinzu und passe auch den zugehörigen Unit-Test entsprechend an. Der findet sich hier:

```
src/test/java/de/dhbw/demo/AssociationsExerciseTest.exercise2()
```